

Prometheus Financial Core — Loan Ledger Edition

Component-Level Design Document

Prepared by: Ledger & Solution Architect Agent

Date: August 16, 2026

Version: 2.0 (Regenerated — reflects the implemented OpenAPI/AsyncAPI contracts)

Inputs: 8 approved epics / 61 REQ-IDs from the Loan Ledger BRD, plus specs/CHANGELOG.md

Status: Implemented and Validated.

Full OpenAPI 3.1 / AsyncAPI 2.6 contracts for all 8 epics now exist under `specs/`, validated with `openapi-spec-validator` and cross-checked against real payloads with `jsonschema`. This document has been regenerated from Version 1.0's pre-implementation design scaffold to match what was actually built — every gap or naming change found during implementation is called out explicitly per epic below, not silently smoothed over. `specs/CHANGELOG.md` remains the authoritative, file-by-file record.

Architecture Context & Conventions

Modules

- **PTY** — Party/CIF (PartyAPI / PartyAdapter)
- **CRM** — CRM (CRMAPI / CRMAdapter — no prior adapter existed in the Core Banking Adapter Layer; drafted fresh: logInteraction, openCase, getCustomer360, zero Money fields anywhere)
- **GL** — General Ledger & Posting Engine (GLPostingAPI / GLPostingAdapter)
- **LAS** — Loan Account Subledger (AccountAPI / AccountAdapter)
- **PAY** — Payment Execution (PaymentAPI / PaymentAdapter)
- **BEOD** — Batch/EOD & Reconciliation (BatchEodAdapter)

Non-Negotiable Constraints

- Each module owns its own data store; cross-module reads use published domain events or a synchronous OpenAPI call to the owning module — never direct database access.
- CRM has no path into the Posting Engine at all. If a design routes a CRM capability through GL, that is a design error, not a shortcut.
- The General Ledger & Posting Engine is the ONLY component permitted to write a JournalEntry. No other module may propose a balance mutation that bypasses it.
- Every JournalEntry is balanced (debits = credits), atomic, immutable, and idempotent (keyed on sourceEventId). There is no update or delete path.
- Every synchronous interface is an OpenAPI 3.1 contract; every asynchronous interface is an AsyncAPI 2.6 contract, before implementation.
- Every capability affecting money movement is gated by a feature flag via FeatureFlagClient, with both an engineering owner and a finance/risk owner named.
- [As-implemented] Money is always {amount: integer minor-units, currency: ISO 4217} — never a bare float — via the shared schema at specs/schemas/money.schema.json.
- [As-implemented] Every balance-affecting operation's idempotency key travels as a required Idempotency-Key header, not a request-body field — except the two batch-trigger operations (runDailyAccrual, runDailyDelinquencyAssessment), which are idempotent by construction on their own (asOf, partitionIndex) body parameters.
- [As-implemented] Every posted JournalEntry line carries runningBalanceAfter (Money) — callers never need a separate read to confirm a posting's effect.
- [As-implemented] Event topics follow "<context>.<entity>.<eventPastTense>", no version suffix (version lives only in each AsyncAPI file's info.version).

Shared Conventions Used Throughout

- GL exposes one generic endpoint, `POST /journal-entries` (idempotent on the Idempotency-Key header, which GL persists as the response's `sourceEventId`), extended per epic with new posting-rule codes

(PR-<EPIC>-<NN>) — never a bespoke posting endpoint per epic. Three mutually-exclusive request shapes now exist: `amount` (single-line rules), `allocation` (fee/interest/principal split against a Cash or Allowance debit), and `capitalization` (interest/fee split against a LoanReceivable debit, CB-MOD only).

- Spec files live under `specs/openapi/*.yaml` (sync), `specs/asynccapi/*.yaml` (async), and `specs/schemas/*.json` (shared objects — Money, Error, Party, JournalEntry, LoanAccount, PaymentInstruction, ServiceCase — referenced via `$ref`, never duplicated inline).
- Cross-module reads are either a synchronous OpenAPI call to the owning module, or a subscription to that module's published event — never a direct database read.
- Money is always `{amount: integer minor-units, currency: ISO 4217}`, never a bare float. Every event topic follows `<context>.<entity>.<eventPastTense>`, no version suffix.

On the receiveRepaymentNotification unification (CB-REPAY, CB-PAYOFF, CB-CHARGEOFF): CB-REPAY, CB-PAYOFF, and CB-CHARGEOFF's recovery flow were each originally designed around PAY publishing an inbound-payment event that AccountAPI subscribes to. During implementation this was retrofitted into a single unified synchronous entry point, AccountAPI's `receiveRepaymentNotification` (POST /repayments:notify) — the API/Event Spec Agent's rules name this explicitly as one of the three canonical balance-affecting operations, which a pure subscribe-side event cannot satisfy (no Idempotency-Key header, no 409 response, is possible on a Kafka consumer). PaymentAPI now calls this endpoint directly and synchronously the moment an inbound payment arrives; the async event (renamed `payment.inbound.received`) still publishes for other consumers and audit, but is no longer what triggers processing. Inside that one endpoint, AccountAPI branches three ways, checked in this order: (1) matched loan account already `ChargedOff` → always a recovery (PR-CHGOFF-02), regardless of any payoff quote reference; (2) a valid, unexpired `payoffQuoteId` present and amount-matched → payoff (PR-PAYOFF-01); (3) otherwise → ordinary repayment waterfall (PR-REPAY-01). This is a real architectural consolidation, not just a rename — the three epics below describe their own trigger step in terms of this shared endpoint rather than three separate flows.

On this document's regeneration: This document was originally written as a pre-implementation design scaffold, before the API/Event Spec Agent authored the actual OpenAPI/AsyncAPI contracts now living under `specs/`. It has been regenerated to match what was actually built — module abbreviations above are kept for readability, each now mapped to its formal Adapter name; interaction sequences reference the real operationIds, event names, and topics; and every gap between the original design notes and the shipped contracts found during implementation is called out explicitly below, not silently smoothed over. `specs/CHANGELOG.md` remains the authoritative, file-by-file record.

Document Coverage

8 design notes · 49 interaction-sequence steps · 11 new posting rules · 33 spec-file changes · 22 domain events · 11 feature flags.

Table of Contents

1. 2. Loan Account Booking [CB-BOOK]
2. 3. Loan Disbursement Processing [CB-DISB]
3. 4. Daily Interest Accrual [CB-ACCR]
4. 5. Loan Repayment Processing [CB-REPAY]

- 5. 6. Delinquency & Past-Due Fee Assessment [CB-DELINQ]
- 6. 7. Early Settlement / Loan Payoff [CB-PAYOFF]
- 7. 8. Loan Charge-off & Write-off [CB-CHARGEOFF]
- 8. 9. Loan Modification (Rate/Term Change) [CB-MOD]

Loan Account Booking [CB-BOOK]

1. Module Ownership

LAS (AccountAPI) owns account creation and the term-version store. PTY (PartyAPI) is queried synchronously for party validation (not read directly). CRM and BEOD are event subscribers only. **GL is not invoked** — booking is intentionally a zero-posting epic.

2. Interaction Sequence

1. Origination system calls **AccountAPI: POST /loan-accounts:book** — idempotent on the **Idempotency-Key** header (value = approvalReferenceId; no longer a request-body field) — REQ-BOOK-001, 004.
2. AccountAPI synchronously calls **PartyAPI: GET /parties/{partyId}** to confirm Active + KYC Verified — REQ-BOOK-002.
3. AccountAPI creates the account (status Approved) with immutable term version v1 (shared LoanAccount/TermVersion schema, principalAmount as Money, rate as annualInterestRateBps) — REQ-BOOK-003, 006.
4. AccountAPI publishes **LoanAccountBooked** on topic **loan.account.booked** — CRM logs an interaction via CRMAPI's **logInteraction** (REQ-BOOK-005); BEOD tracks it for the approvals-vs-bookings check (REQ-BOOK-007).
5. Rejection path (malformed terms or failed party validation): synchronous 400/422, AND AccountAPI publishes **LoanBookingRejected** on topic **loan.booking.rejected** to BEOD's exception queue (REQ-BOOK-001's explicit "exception queue" language) — present in the shipped contract but absent from the original design note's domain-events table; added here to match.
6. **No Posting Engine call at any step** — a deliberate absence, not an oversight.

Posting-Rule Impact

No posting rules — this epic makes zero Posting Engine calls by design.

3. Spec Files (New / Modified)

File	Purpose
specs/openapi/loan-account-subledger.yaml	POST /loan-accounts:book, GET /loan-accounts/{id}
specs/asynccapi/loan-account-subledger-events.yaml	channels loan.account.booked, loan.booking.rejected
specs/schemas/loan-account.schema.json	shared TermSet/TermVersion/LoanAccount, referenced via \$ref
specs/schemas/party.schema.json	shared Party object returned by PartyAPI

4. Domain Events

Event	Producer	Consumer(s)
LoanAccountBooked (<code>loan.account.booked</code>)	AccountAdapter	CRMAdapter, BatchEodAdapter
LoanBookingRejected (<code>loan.booking.rejected</code>)	AccountAdapter	BatchEodAdapter

5. Feature Flags

Key	Type	Rollout Plan	Owners
<code>loan.booking.enabled</code>	boolean kill-switch	Test-party dark launch → 10% of approval volume → 100%	Eng: Loan Subledger lead Risk: Not required — epic moves no money

6. NFR & Security Implications

- Synchronous PartyAPI dependency needs a timeout + fail-closed circuit breaker (reject booking rather than silently skip KYC on timeout).
- Events carry partyId only, no PII payload.
- Term-version creation is logged with actor + approvalReferenceId for exam trail.
- Adds one new BEOD reconciliation job (approvals vs. bookings).
- [As-implemented] No Idempotency-Key is required by rule for this operation strictly (booking posts no balance), but the retrofit applies the header uniformly to every creation-style operation for consistency — see the Idempotency-Key convention note in Architecture Context.

2 Loan Disbursement Processing [CB-DISB]

1. Module Ownership

LAS (AccountAPI) owns the disbursement state machine and orchestrates PTY/GL/PAY calls. GL (GLPostingAPI) owns the funding posting. PAY (PaymentAPI) owns payment execution. CRM/BEOD are subscribers.

2. Interaction Sequence

1. AccountAPI: POST `/loan-accounts/{id}/disbursements` — Idempotency-Key header (value = `disbursementId`), status → Pending Disbursement — REQ-DISB-001.
2. AccountAPI → PartyAPI: GET `/parties/{partyId}`; reject with reason code on failure — REQ-DISB-002.
3. AccountAPI → GLPostingAPI: POST `/journal-entries`, rule **PR-DISB-01** (Dr LoanReceivable P / Cr Cash-Nostro P), Idempotency-Key = `disbursementId` — REQ-DISB-003. Response now includes `runningBalanceAfter` (Money) per line.
4. On GL confirmation, AccountAPI → PaymentAPI: POST `/payment-instructions:disburse` (operationId `initiatedDisbursement` — renamed from the original `disbursePayment/{id}:disburse` draft), Idempotency-Key = `disbursementId` — REQ-DISB-004. Response is the shared `PaymentInstruction` object.
5. `payment.disbursement.confirmed` (renamed from `PaymentExecuted/payment.executed.v1`) → AccountAPI sets status Disbursed, publishes `LoanAccountDisbursed` on `loan.account.disbursed` → CRM (REQ-DISB-005), BEOD (REQ-DISB-006).
6. `payment.disbursement.failed` (renamed from `PaymentFailed/payment.failed.v1`) → AccountAPI → GL posts **PR-DISB-02** (reversal of PR-DISB-01, new Idempotency-Key = `disbursementId:reverse`), status reverts to Approved only after GL confirms — REQ-DISB-007.

Posting-Rule Impact

Rule ID	Entry	Trigger
PR-DISB-01	Dr LoanReceivable P / Cr Cash-Nostro P	confirmed disbursement request, post party validation
PR-DISB-02	reversal of PR-DISB-01	confirmed payment failure/return

3. Spec Files (New / Modified)

File	Purpose
<code>specs/openapi/loan-account-subledger.yaml</code>	POST/GET <code>.../disbursements</code> , POST <code>.../disbursements/{id}:reverse</code>

specs/openapi/gl-posting-engine.yaml	register PR-DISB-01, PR-DISB-02
specs/openapi/payment-execution.yaml	POST /payment-instructions:disburse, GET /payment-instructions/{id}
specs/asynapi/payment-execution-events.yaml	channels payment.disbursement.confirmed, payment.disbursement.failed
specs/schemas/payment-instruction.schema.json	shared PaymentInstruction object
specs/schemas/journal-entry.schema.json	shared JournalEntry response with runningBalanceAfter

4. Domain Events

Event	Producer	Consumer(s)
JournalEntryPosted (gl.entry.posted, filtered PR-DISB-*)	GLPostingAdapter	AccountAdapter, BatchEodAdapter
PaymentDisbursementConfirmed / PaymentDisbursementFailed	PaymentAdapter	AccountAdapter
LoanAccountDisbursed	AccountAdapter	CRMAAdapter, BatchEodAdapter

5. Feature Flags

Key	Type	Rollout Plan	Owners
loan.disbursement.enabled	boolean kill-switch	Shadow-post (compute, no PaymentAPI call) 1 week → 5% live volume → 25% → 100%, auto-rollback on reversal-rate SLO breach	Eng: Loan Subledger lead Risk: Treasury Ops lead

6. NFR & Security Implications

- GL post target p95 < 200ms; PaymentAPI call acks async (202) with status updates via event.
- Borrower account/routing details stay inside PaymentAPI, never in the disbursement event payload.
- Every state transition logged with correlationId=disbursementId.
- New BEOD daily total-variance check (REQ-DISB-006).
- [As-implemented] gl.entry.posted's per-line runningBalanceAfter means BEOD's daily variance check no longer needs a second read to confirm each posting's effect on the loan account balance.

3 Daily Interest Accrual [CB-ACCR]

1. Module Ownership

BEOD orchestrates the daily run; LAS (AccountAPI) computes eligibility and the accrual amount (resolving the effective term version — see CB-MOD dependency); GL posts. No CRM touchpoint in this epic.

2. Interaction Sequence

1. BEOD triggers the accrual job for date D.
2. BEOD → AccountAPI: GET `/loan-accounts:accrual-eligible?asOf=D` — REQ-ACCR-001, 005.
3. BEOD → AccountAPI: POST `/loan-accounts:accrue` — the actual batch-execution trigger. **Gap fixed vs. the original design note:** the note only ever specified the eligibility GET; nothing triggered per-account computation and posting. This endpoint (no Idempotency-Key header — idempotent by construction on its own `asOf/partitionIndex` body) does that: for each eligible account it resolves rate/day-count/principal-basis from the term version effective as of D (REQ-CB-MOD-006), computes X, and posts PR-ACCR-01.
4. AccountAPI → GLPostingAPI: POST `/journal-entries`, rule **PR-ACCR-01** (Dr InterestReceivable X / Cr InterestIncome X), Idempotency-Key = `accr:{loanAccountId}:{D}` — REQ-ACCR-002, 003. metadata carries `businessDate/annualInterestRateBps/dayCountConvention/principalBasis/termVersion` for audit reproducibility (REQ-ACCR-004).
5. BEOD cross-checks eligible-account count vs. posted-entry count (via `gl.entry.posted` filtered to PR-ACCR-01) at batch close, raises `accrual.exception.raised` on mismatch — REQ-ACCR-007.

Posting-Rule Impact

Rule ID	Entry	Trigger
PR-ACCR-01	Dr InterestReceivable X / Cr InterestIncome X	daily eligible-account evaluation; idempotent on (loanAccountId, businessDate)

3. Spec Files (New / Modified)

File	Purpose
<code>specs/openapi/loan-account-subledger.yaml</code>	GET <code>.../accrual-eligible</code> , POST <code>/loan-accounts:accrue</code> (the batch-trigger action the original design note omitted)
<code>specs/openapi/gl-posting-engine.yaml</code>	register PR-ACCR-01; metadata field added to <code>PostJournalEntryRequest/JournalEntry</code> for audit context

specs/asynccapi/batch-eod-events.yaml	channel accrual.exception.raised (renamed from accrual.exception.raised.v1)
---------------------------------------	---

4. Domain Events

Event	Producer	Consumer(s)
JournalEntryPosted (gl.entry.posted, filtered PR-ACCR-01)	GLPostingAdapter	BatchEodAdapter
AccrualExceptionRaised (accrual.exception.raised)	BatchEodAdapter	Ops/Financial-Reporting (human, not a module — see x-consumers: [] in the AsyncAPI file)

5. Feature Flags

Key	Type	Rollout Plan	Owners
loan.accrual.batch.enabled	Boolean kill-switch	Shadow-compute one full month-end cycle (no posting) → post for a pilot cohort → 100%	Eng: Batch/EOD lead Risk: Controller / Financial Reporting owner

6. NFR & Security Implications

- Must complete within the overnight window — both accrual-eligible and accrue support partitionIndex/partitionCount fan-out, operationalizing this NFR directly in the contract rather than leaving it as prose.
- Every PR-ACCR-01 entry retains rate/day-count/principal as structured metadata for exam reproducibility (REQ-ACCR-004).
- This epic is itself a reconciliation control; no separate downstream recon needed.
- [As-implemented] Rate resolution now goes through the effective-dated term-version lookup CB-MOD introduces, not a single static rate field — see the receiveRepaymentNotification-adjacent cross-epic note and Appendix A.

4 Loan Repayment Processing [CB-REPAY]

1. Module Ownership

PAY (PaymentAPI) receives the inbound payment and calls AccountAPI synchronously; LAS (AccountAPI) owns matching and waterfall allocation; GL posts; CRM logs. **See the receiveRepaymentNotification unification note in Architecture Context** — this epic's trigger step changed materially from the original design.

2. Interaction Sequence

1. PaymentAPI receives an inbound payment and calls `AccountAPI: POST /repayments:notify` (operationId `receiveRepaymentNotification`) synchronously — Idempotency-Key header = `paymentReferenceId`. This REPLACES the original "PAY publishes PaymentReceived, LAS subscribes" design; the async event (renamed `payment.inbound.received`) still publishes, now for observability/audit only.
2. AccountAPI matches `loanAccountRef` to exactly one active loan account — REQ-REPAY-001; ambiguous/no match publishes `payment.match.failed` (renamed from `PaymentUnmatched`) to BEOD's exception queue, and returns a Repayment with status `Unmatched` — still 201/200, not an error.
3. Not a payoff, not a recovery (see the three-way branch in the unification note): AccountAPI computes the waterfall (fees → interest → principal) — REQ-REPAY-002.
4. AccountAPI → GLPostingAPI: `POST /journal-entries`, rule **PR-REPAY-01** (Dr Cash/Nostro P / Cr FeeReceivable + InterestReceivable + LoanReceivable per `allocation` — clearing receivables already recognized at accrual/assessment time, never re-crediting Income) — REQ-REPAY-003, 004.
5. AccountAPI publishes `LoanRepaymentPosted` on `loan.repayment.posted` → CRM — REQ-REPAY-005.
6. BEOD cross-checks all received payments against posted/unmatched/reversed outcomes, raising `repayment.exception.raised` for anything left in a non-terminal state at EOD cutoff — REQ-REPAY-006.
7. Reversal: AccountAPI: `POST /repayments/{id}:reverse` (Idempotency-Key = `{repaymentId}:reverse`) → GL posts **PR-REPAY-02** (offsetting); misapplied case re-runs the waterfall against the correct account as a fresh PR-REPAY-01 under a new Idempotency-Key (`{repaymentId}:correction`) — REQ-REPAY-007.

Posting-Rule Impact

Rule ID	Entry	Trigger
PR-REPAY-01	Dr Cash/Nostro P / Cr FeeReceivable + InterestReceivable + LoanReceivable (per allocation)	matched, waterfall-allocated repayment — clears receivables built up by PR-ACCR-01/PR-DELINQ-01, never re-credits Income
PR-REPAY-02	reversal of PR-REPAY-01	confirmed return (NSF) or misapplication

3. Spec Files (New / Modified)

File	Purpose
specs/asyncapi/payment-execution-events.yaml	channel payment.inbound.received (renamed from payment.received.v1; observability-only now, not the trigger)
specs/openapi/loan-account-subledger.yaml	POST /repayments:notify (receiveRepaymentNotification — the new authoritative synchronous trigger), GET /repayments/{id}, POST /repayments/{id}:reverse
specs/openapi/gl-posting-engine.yaml	register PR-REPAY-01, PR-REPAY-02; Allocation schema (categorized fee/interest/principal split, Money-based)
specs/asyncapi/loan-account-subledger-events.yaml	channel payment.match.failed (renamed from payment.unmatched.v1), loan.repayment.posted
specs/asyncapi/batch-eod-events.yaml	channel repayment.exception.raised — present in the shipped contract but missing from the original design note's domain-events table; added here to match

4. Domain Events

Event	Producer	Consumer(s)
PaymentInboundReceived (payment.inbound.received)	PaymentAdapter	BatchEodAdapter (observability only — no longer drives AccountAdapter processing)
PaymentMatchFailed (payment.match.failed)	AccountAdapter	BatchEodAdapter
LoanRepaymentPosted (loan.repayment.posted)	AccountAdapter	CRMAdapter
RepaymentExceptionRaised (repayment.exception.raised)	BatchEodAdapter	Ops/Financial-Reporting (human)

5. Feature Flags

Key	Type	Rollout Plan	Owners
-----	------	--------------	--------

loan.repayment.autopost. boolean	loan.repayment.autopost. boolean → kill-switch	Dual-run vs. legacy/manual posting 2 weeks (compare only) → cut over per payment rail, ACH first → all rails	Eng: Loan Subledger lead Risk: Cash Ops/Treasury lead
---	---	---	---

6. NFR & Security Implications

- Match→allocate→post should complete within seconds to hold same-day posting SLA — now a hard synchronous-call latency budget, not just an async-processing target, since PaymentAPI blocks on receiveRepaymentNotification's response.
- PaymentInboundReceived/receiveRepaymentNotification carry reference IDs only, not payer bank details.
- Waterfall breakdown stored as line-item metadata for dispute resolution.
- Unmatched-payment queue needs its own alerting SLA, not just EOD visibility.
- Income-recognition principle: PR-REPAY-01 clears Cash against Fee/Interest/Loan Receivable — it never credits Fee/Interest Income directly, since that income was already recognized once at accrual (PR-ACCR-01) or assessment (PR-DELINQ-01) time. Crediting Income again here would double-count it. (This was a real documentation error in this design note's first draft, corrected during the API/Event Spec Agent implementation pass — see specs/CHANGELOG.md's v0.3.0 entry.)

5 Delinquency & Past-Due Fee Assessment [CB-DELINQ]

1. Module Ownership

BEOD orchestrates; LAS (AccountAPI) owns DPD/fee computation and the Non-Accrual flag; GL posts; CRM logs.

2. Interaction Sequence

1. BEOD triggers the delinquency job for date D.
2. BEOD → AccountAPI: GET /loan-accounts:past-due?asOf=D — REQ-DELINQ-001.
3. BEOD → AccountAPI: POST /loan-accounts:assess-delinquency — the actual batch-execution trigger. **Gap fixed vs. the original design note:** same omission pattern as CB-ACCR — the note specified only the read, not the trigger. No Idempotency-Key header (idempotent by construction on asOf/partitionIndex). Updates DPD count/bucket for every account EXCLUDING those already ChargedOff (REQ-CHARGEOFF-005) — REQ-DELINQ-003.
4. Past-grace-period accounts: AccountAPI → GLPostingAPI: POST /journal-entries, rule **PR-DELINQ-01** (Dr FeeReceivable / Cr FeeIncome), Idempotency-Key = late-fee:{loanAccountId}:{scheduledDueDate} — REQ-DELINQ-002, 004.
5. DPD threshold crossing → AccountAPI sets LoanAccount.nonAccrualFlag, publishes LoanNonAccrualFlagged (renamed from LoanAccountNonAccrualFlagged) on loan.nonaccrual.flagged — read by **CB-ACCR's eligibility query** as a same-service internal lookup, not via the event, which exists for BatchEodAdapter visibility/audit only — REQ-DELINQ-005 (cross-epic dependency, not a new posting).
6. AccountAPI publishes DelinquencyStatusChanged on delinquency.status.changed → CRM — REQ-DELINQ-006.
7. BEOD cross-checks fee postings vs. DPD changes, raising delinquency.exception.raised on mismatch in either direction — REQ-DELINQ-008. **Present in the shipped contract but missing from the original design note's domain-events table;** added here to match.
8. Waiver: AccountAPI: POST /fees/{id}:waive (Idempotency-Key = {feeId}:waive) with mandatory confirmedBy/reasonCode → GL posts **PR-DELINQ-02** (reversal), metadata carrying the same authorization pair so it travels with the ledger entry, not just AccountAPI's own record — REQ-DELINQ-007.

Posting-Rule Impact

Rule ID	Entry	Trigger
PR-DELINQ-01	Dr FeeReceivable / Cr FeeIncome	account past grace period on a missed installment
PR-DELINQ-02	reversal of PR-DELINQ-01	authorized fee waiver

3. Spec Files (New / Modified)

File	Purpose
specs/openapi/loan-account-subledger.yaml	GET .../past-due, POST /loan-accounts:assess-delinquency (batch trigger the original note omitted), GET/POST /fees/{id}[:waive]
specs/openapi/gl-posting-engine.yaml	register PR-DELINQ-01, PR-DELINQ-02; mandatory metadata (waivedBy/reasonCode) enforced for PR-DELINQ-02
specs/asyncapi/loan-account-subledger-events.yaml	channels delinquency.status.changed, loan.nonaccrual.flagged (both renamed, .v1 dropped)
specs/asyncapi/batch-eod-events.yaml	channel delinquency.exception.raised — added to match the shipped contract

4. Domain Events

Event	Producer	Consumer(s)
DelinquencyStatusChanged (delinquency.status.changed)	AccountAdapter	CRMAdapter
LoanNonAccrualFlagged (loan.nonaccrual.flagged)	AccountAdapter	BatchEodAdapter (visibility only — CB-ACCR reads the flag same-service, not via this event)
DelinquencyExceptionRaised (delinquency.exception.raised)	BatchEodAdapter	Ops/Financial-Reporting (human)

5. Feature Flags

Key	Type	Rollout Plan	Owners
loan.delinquency.feeassessment.enabled	boolean, on-switch	Shadow-compute 1 cycle → post for one cohort → all	Eng: Loan Subledger lead Risk: Collections/Risk lead
loan.delinquency.feewaive.enabled	boolean, RBAC-gated	Enabled only for authorized Ops role from day one, no volume ramp	Eng: Loan Subledger lead Risk: Collections/Risk lead

6. NFR & Security Implications

- Same partitioning concern as Accrual — assess-delinquency supports the same partitionIndex/partitionCount fan-out.
- Waiver requires a mandatory (not optional) authorization record on PR-DELINQ-02 — enforced in GL's metadata requirement, not just AccountAPI's own record.
- Sequencing dependency: CB-ACCR's daily step must run after this epic's Non-Accrual flag update in the BEOD DAG, or accrual uses stale eligibility.

6 Early Settlement / Loan Payoff [CB-PAYOFF]

1. Module Ownership

LAS (AccountAPI) owns quote generation and closure; GL posts; PAY receives the payoff payment and routes it through the unified receiveRepaymentNotification entry point (see Architecture Context); CRM logs; BEOD reconciles.

2. Interaction Sequence

1. AccountAPI: GET /loan-accounts/{id}/payoff-quote?goodThrough=G — read-only, no posting, returns a PayoffQuote with a quoteId — REQ-PAYOFF-001, 002, 003.
2. Payoff payment arrives via PaymentAPI, which calls AccountAPI: POST /repayments:notify with payoffQuoteId set — the SAME endpoint CB-REPAY uses, not a separate settlement call (see the receiveRepaymentNotification unification note).
3. AccountAPI validates amount/window against the quote — exact match proceeds to Branch (b) of the unified endpoint; underpayment degrades to the standard waterfall (CB-REPAY reuse, PR-REPAY-01); overpayment posts PR-PAYOFF-01 for the exact quote total and routes the excess to Payoff.suspenseAmount — REQ-PAYOFF-003, 007. An expired/not-found quote also degrades to the standard repayment path, since there is no caller left to reject to once funds have arrived.
4. AccountAPI → GLPostingAPI: POST /journal-entries, rule **PR-PAYOFF-01** (Dr Cash/Nostro / Cr LoanReceivable + InterestReceivable via allocation, zeroing both — never crediting Income) — REQ-PAYOFF-004, 005.
5. On confirmation, AccountAPI sets status Closed — REQ-PAYOFF-006 — and publishes LoanAccountClosed on loan.account.closed → CRM (REQ-PAYOFF-008), BEOD.
6. BEOD confirms true zero balance on every account closed that day, raising payoff.exception.raised on any residual — REQ-PAYOFF-009. **Present in the shipped contract but missing from the original design note's domain-events table**; added here to match.

Posting-Rule Impact

Rule ID	Entry	Trigger
PR-PAYOFF-01	Dr Cash/Nostro / Cr LoanReceivable + InterestReceivable (zeroing both, via allocation)	payoff payment matching a valid, unexpired quote, processed through receiveRepaymentNotification

3. Spec Files (New / Modified)

File	Purpose
------	---------

specs/openapi/loan-account-subledger.yaml	GET .../payoff-quote, GET /payoffs/{id}; receiveRepaymentNotification extended with the payoffQuoteId branch (no separate settlement endpoint)
specs/openapi/gl-posting-engine.yaml	register PR-PAYOFF-01
specs/asyncapi/loan-account-subledger-events.yaml	channel loan.account.closed (renamed from loan.account.closed.v1)
specs/asyncapi/batch-eod-events.yaml	channel payoff.exception.raised — added to match the shipped contract

4. Domain Events

Event	Producer	Consumer(s)
LoanAccountClosed (loan.account.closed)	AccountAdapter	CRMAAdapter, BatchEodAdapter
JournalEntryPosted (gl.entry.posted, filtered PR-PAYOFF-01)	GLPostingAdapter	AccountAdapter, BatchEodAdapter
PayoffExceptionRaised (payoff.exception.raised)	BatchEodAdapter	Ops/Financial-Reporting (human)

5. Feature Flags

Key	Type	Rollout Plan	Owners
loan.payoff.quote.enabled	boolean, no money movement	Broad early enablement — read-only	Eng: Loan Subledger lead Risk: Not required — quote step posts nothing
loan.payoff.settlement.enabled	boolean, gates PR-PAYOFF-01 + closure	Ramp 10% → 50% → 100% of payoff volume	Eng: Loan Subledger lead Risk: Treasury/Controller

6. NFR & Security Implications

- Quote generation must be near-real-time (borrower/CSR-facing).
- Quote itemization is retained and linked to the settling journal entry for dispute traceability.
- Adds REQ-PAYOFF-009 to the BEOB daily job catalog.

- There is no reversal endpoint for a Payoff in the shipped contract — a returned/NSF payoff payment is not yet a specified flow. Open question carried forward unresolved, same class as the analogous gap noted under CB-CHARGEOFF's recovery.

7 Loan Charge-off & Write-off [CB-CHARGEOFF]

1. Module Ownership

LAS (AccountAPI) owns the state machine; recovery is routed through the unified receiveRepaymentNotification entry point (see Architecture Context); GL posts; CRM logs; BEOD reconciles the reserve account via a new portfolio-wide GL balance read.

2. Interaction Sequence

1. Collections system calls AccountAPI: `POST /loan-accounts/{id}:chargeoff` (Idempotency-Key = chargeoffDecisionReference) — REQ-CHGOFF-001, status Pending Charge-off.
2. AccountAPI → GLPostingAPI: `POST /journal-entries`, rule **PR-CHGOFF-01** (Dr AllowanceForLoanLosses (B+I) / Cr LoanReceivable B, InterestReceivable I, via `allocation`), metadata mandatorily carrying chargeoffDecisionReference/confirmedBy — REQ-CHGOFF-002, 004.
3. On confirmation, status → Charged-Off — REQ-CHGOFF-003. AccountAPI publishes `LoanAccountChargedOff` on `loan.account.chargedoff` → CRM (REQ-CHGOFF-007); the ChargedOff status is also read directly, same-service, by **CB-ACCR** and **CB-DELINQ**'s eligibility queries to exclude the account — REQ-CHGOFF-005 (same cross-epic pattern as the Non-Accrual flag).
4. Recovery: PaymentAPI receives the inbound payment and calls AccountAPI: `POST /repayments:notify` — same unified endpoint as CB-REPAY/CB-PAYOFF. Because the matched loan account is already ChargedOff, this branch is checked FIRST, ahead of the repayment/payoff branches, regardless of any payoffQuoteId present. Posts **PR-CHGOFF-02** (Dr Cash/Nostro / Cr RecoveryIncome) — REQ-CHGOFF-006; status remains Charged-Off — this is **not** a reversal of PR-CHGOFF-01.
5. BEOD → GLPostingAPI: `GET /gl-accounts/AllowanceForLoanLosses/balance` — a new, portfolio-wide GL capability distinct from per-loan-account runningBalanceAfter, added specifically because this reconciliation needed it — and compares against the running sum of PR-CHGOFF-01 postings (via `gl.entry.posted`), raising `chargeoff.exception.raised` on mismatch — REQ-CHGOFF-008.

Posting-Rule Impact

Rule ID	Entry	Trigger
PR-CHGOFF-01	Dr AllowanceForLoanLosses (B+I) / Cr LoanReceivable B, InterestReceivable I	confirmed, approved charge-off decision
PR-CHGOFF-02	Dr Cash/Nostro / Cr RecoveryIncome	recovery payment on a charged-off account (distinct entry type, not a reversal)

3. Spec Files (New / Modified)

File	Purpose
specs/openapi/loan-account-subledger.yaml	POST .../chargeoff, GET /chargeoffs/{id}, GET /recoveries/{id}; receiveRepaymentNotification extended with the ChargedOff-status branch
specs/openapi/gl-posting-engine.yaml	register PR-CHGOFF-01 (first rule debiting a non-Cash control account), PR-CHGOFF-02; NEW GET /gl-accounts/{code}/balance
specs/asyncapi/loan-account-subledger-events.yaml	channel loan.account.chargedoff (renamed from loan.account.chargedoff.v1)
specs/asyncapi/batch-eod-events.yaml	channel chargeoff.exception.raised — a fundamentally different aggregation pattern (portfolio-wide amount comparison, not per-date count comparison) than the other three exception channels

4. Domain Events

Event	Producer	Consumer(s)
LoanAccountChargedOff (loan.account.chargedoff)	AccountAdapter	CRMAAdapter, BatchEodAdapter
JournalEntryPosted (gl.entry.posted, filtered PR-CHGOFF-01/02)	GLPostingAdapter	AccountAdapter, BatchEodAdapter
ChargeoffExceptionRaised (chargeoff.exception.raised)	BatchEodAdapter	Ops/Financial-Reporting (human)

5. Feature Flags

Key	Type	Rollout Plan	Owners
loan.chargeoff.enabled	boolean kill-switch	Pilot on one collections cohort with manual pre/post recon review each cycle → expand	Eng: Loan Subledger lead Risk: Controller / Allowance-for-Losses owner

6. NFR & Security Implications

- Not latency-sensitive (batch/manual-triggered process).

- chargeoffDecisionReference must retain a durable link to the upstream credit-policy approval artifact — retained verbatim on PR-CHGOFF-01's metadata, not just AccountAPI's own ChargeOff record.
- Reserve reconciliation aggregates across the entire portfolio against one GL control account — a different aggregation pattern than every other epic's per-date, per-account check; this is why it needed a new GET /gl-accounts/{code}/balance capability rather than reusing gl.entry.posted alone.
- No reversal endpoint exists for a Recovery — same class of open question as CB-PAYOFF's missing payoff-reversal flow.

8 Loan Modification (Rate/Term Change) [CB-MOD]

1. Module Ownership

LAS (AccountAPI) owns term-versioning and the conditional posting branch; GL posts only for balance-impacting modifications; CRM logs.

2. Interaction Sequence

1. AccountAPI: `POST /loan-accounts/{id}/modifications` (operationId `applyModification`, Idempotency-Key = `modificationReferenceId`) with new terms + effective date E — REQ-MOD-001, 005.
2. AccountAPI appends immutable term version `v_n+1` effective E; prior versions untouched — REQ-MOD-002.
3. **Branch A (capitalization):** AccountAPI → GLPostingAPI: `POST /journal-entries`, rule **PR-MOD-01** (Dr LoanReceivable X / Cr InterestReceivable and/or FeeReceivable X, via a new `capitalization` input shape — distinct from `allocation`, since principal is the debit here, not a third credit category) — REQ-MOD-003.
4. **Branch B (rate/term only):** no GL call is made — REQ-MOD-004; implemented as a true no-op, not a zero-amount journal entry.
5. AccountAPI publishes `LoanTermsModified` on `loan.terms.modified` → CRM (REQ-MOD-007) — fires for BOTH branches, unlike this catalog's exception-queue events which fire for only one side of a decision.
6. `GET /loan-accounts/{id}/term-versions` (full, unfiltered) serves the audit requirement — REQ-MOD-008. **Split from the original single combined-endpoint design** into two operations: this one for full history, and a second, `GET /loan-accounts/{id}/term-versions:effective?asOf=D`, for the single-version point-in-time resolution CB-ACCR's rate lookup performs internally (REQ-MOD-006).

Posting-Rule Impact

Rule ID	Entry	Trigger
PR-MOD-01	Dr LoanReceivable X / Cr InterestReceivable and/or FeeReceivable X	approved modification that capitalizes past-due interest/fees (conditional — Branch A only)

3. Spec Files (New / Modified)

File	Purpose
<code>specs/openapi/loan-account-subledger.yaml</code>	<code>POST .../modifications</code> , <code>GET /modifications/{id}</code> , <code>GET .../term-versions</code> , <code>GET .../term-versions:effective</code> (split into two operations vs. the original single-endpoint design)

specs/openapi/gl-posting-engine.yaml	register PR-MOD-01; NEW Capitalization schema/oneOf branch on PostJournalEntryRequest (3-way exclusive choice with amount/allocation)
specs/asyncapi/loan-account-subledger-events.yaml	channel loan.terms.modified (renamed from loan.terms.modified.v1)

4. Domain Events

Event	Producer	Consumer(s)
LoanTermsModified (loan.terms.modified)	AccountAdapter	CRMAAdapter
JournalEntryPosted (gl.entry.posted, filtered PR-MOD-01)	GLPostingAdapter	AccountAdapter, BatchEodAdapter — fires only for Branch A

5. Feature Flags

Key	Type	Rollout Plan	Owners
loan.modification.enabled	Boolean, gates term-versioning broadly (Branch B has no money movement)	Enable rate/term-only path on a small cohort first; validate accrual correctly switches versions across the effective-date boundary before touching Branch A	Eng: Loan Subledger lead Risk: Not required for the Branch-B path
loan.modification.capitalization.enabled	Boolean, gates PR-MOD-01 specifically	Ramp 10% → 50% → 100% of approved capitalization modifications, with reconciliation review each step	Eng: Loan Subledger lead Risk: Controller

6. NFR & Security Implications

- Not latency-sensitive.
- Term-version history must be retained indefinitely with no purge/TTL — confirm explicitly against the generic operational-log retention policy, which may default shorter.
- Breaking-change dependency: CB-MOD changes what CB-ACCR reads (a single static rate record → a versioned lookup); this dependency shipped correctly — CB-ACCR's runDailyAccrual description now explicitly references the effective-dated resolution.

- [Bug caught during implementation] The first draft of `Modification.capitalizedAmount` combined a nullable outer type with `allOf`-referencing `Money`, which is a genuine schema contradiction (`Money`'s own `type:object` constraint would reject null under `allOf` regardless of the outer type union). Fixed to `oneOf[Money, null]` and verified against real payloads — see `specs/CHANGELOG.md`.

Appendix A — Cross-Epic Dependency Note

Three epics — CB-DELINQ (Non-Accrual flag), CB-CHARGEOFF (Charged-Off status), and CB-MOD (term-version lookup) — all modify what CB-ACCR reads at daily-batch time, and all three resolve as same-service internal reads inside AccountAPI, not cross-service event subscriptions — CB-ACCR does not subscribe to `loan.nonaccrual.flagged` or `loan.account.chargedoff` to drive its own eligibility logic; those events exist for BatchEodAdapter visibility only. None of this is a scope problem, but the BEOD DAG needs explicit sequencing: CB-DELINQ and CB-CHARGEOFF's status updates must run before CB-ACCR's eligibility query in the daily batch order, and CB-MOD's term-version data must exist before CB-ACCR's rate resolution runs against it. This is a build-order constraint for engineering, not a scope gap — and it shipped correctly: `runDailyAccrual`'s implemented description references all three dependencies explicitly.

Appendix B — Consolidated Posting-Rule Catalog

Every journal entry this design introduces, across all 8 epics. GL is the only writer for all of these.

Rule ID	Epic	Entry	Trigger
PR-DISB-01	CB-DISB	Dr LoanReceivable P / Cr Cash-Nostro P	confirmed disbursement request, post party validation
PR-DISB-02	CB-DISB	reversal of PR-DISB-01	confirmed payment failure/return
PR-ACCR-01	CB-ACCR	Dr InterestReceivable X / Cr InterestIncome X	daily eligible-account evaluation; idempotent on (loanAccountld, businessDate)
PR-REPAY-01	CB-REPAY	Dr Cash/Nostro P / Cr FeeReceivable + InterestReceivable + LoanReceivable (per allocation)	matched, waterfall-allocated repayment — clears receivables built up by PR-ACCR-01/PR-DELINQ-01, never re-credits Income
PR-REPAY-02	CB-REPAY	reversal of PR-REPAY-01	confirmed return (NSF) or misapplication
PR-DELINQ-01	CB-DELINQ	Dr FeeReceivable / Cr FeeIncome	account past grace period on a missed installment
PR-DELINQ-02	CB-DELINQ	reversal of PR-DELINQ-01	authorized fee waiver
PR-PAYOFF-01	CB-PAYOFF	Dr Cash/Nostro / Cr LoanReceivable + InterestReceivable (zeroing both, via allocation)	payoff payment matching a valid, unexpired quote, processed through receiveRepaymentNotification
PR-CHG0FF-01	CB-CHARGE0FF	Dr AllowanceForLoanLosses (B+I) / Cr LoanReceivable B, InterestReceivable I	confirmed, approved charge-off decision

PR-CHGOFF-02	CB-CHARGEOFF	Dr Cash/Nostro / Cr RecoveryIncome	recovery payment on a charged-off account (distinct entry type, not a reversal)
PR-MOD-01	CB-MOD	Dr LoanReceivable X / Cr InterestReceivable and/or FeeReceivable X	approved modification that capitalizes past-due interest/fees (conditional — Branch A only)

Appendix C — Consolidated Domain Event Catalog

Event	Epic	Producer	Consumer(s)
LoanAccountBooked (loan.account.booked)	CB-BOOK	AccountAdapter	CRMAdapter, BatchEodAdapter
LoanBookingRejected (loan.booking.rejected)	CB-BOOK	AccountAdapter	BatchEodAdapter
JournalEntryPosted (gl.entry.posted, filtered PR-DISB-*)	CB-DISB	GLPostingAdapter	AccountAdapter, BatchEodAdapter
PaymentDisbursementConfirmed / PaymentDisbursementFailed	CB-DISB	PaymentAdapter	AccountAdapter
LoanAccountDisbursed	CB-DISB	AccountAdapter	CRMAdapter, BatchEodAdapter
JournalEntryPosted (gl.entry.posted, filtered PR-ACCR-01)	CB-ACCR	GLPostingAdapter	BatchEodAdapter
AccrualExceptionRaised (accrual.exception.raised)	CB-ACCR	BatchEodAdapter	Ops/Financial-Reporting (human, not a module — see x-consumers: [] in the AsyncAPI file)
PaymentInboundReceived (payment.inbound.received)	CB-REPAY	PaymentAdapter	BatchEodAdapter (observability only — no longer drives AccountAdapter processing)
PaymentMatchFailed (payment.match.failed)	CB-REPAY	AccountAdapter	BatchEodAdapter
LoanRepaymentPosted (loan.repayment.posted)	CB-REPAY	AccountAdapter	CRMAdapter
RepaymentExceptionRaised (repayment.exception.raised)	CB-REPAY	BatchEodAdapter	Ops/Financial-Reporting (human)

DelinquencyStatusChanged (delinquency.status.changed)	CB-DELINQ	AccountAdapter	CRMAdapter
LoanNonAccrualFlagged (loan.nonaccrual.flagged)	CB-DELINQ	AccountAdapter	BatchEodAdapter (visibility only — CB-ACCR reads the flag same-service, not via this event)
DelinquencyExceptionRaised (delinquency.exception.raised)	CB-DELINQ	BatchEodAdapter	Ops/Financial-Reporting (human)
LoanAccountClosed (loan.account.closed)	CB-PAYOFF	AccountAdapter	CRMAdapter, BatchEodAdapter
JournalEntryPosted (gl.entry.posted, filtered PR-PAYOFF-01)	CB-PAYOFF	GLPostingAdapter	AccountAdapter, BatchEodAdapter
PayoffExceptionRaised (payoff.exception.raised)	CB-PAYOFF	BatchEodAdapter	Ops/Financial-Reporting (human)
LoanAccountChargedOff (loan.account.chargedoff)	CB-CHARGEOFF	AccountAdapter	CRMAdapter, BatchEodAdapter
JournalEntryPosted (gl.entry.posted, filtered PR-CHGOFF-01/02)	CB-CHARGEOFF	GLPostingAdapter	AccountAdapter, BatchEodAdapter
ChargeoffExceptionRaised (chargeoff.exception.raised)	CB-CHARGEOFF	BatchEodAdapter	Ops/Financial-Reporting (human)
LoanTermsModified (loan.terms.modified)	CB-MOD	AccountAdapter	CRMAdapter
JournalEntryPosted (gl.entry.posted, filtered PR-MOD-01)	CB-MOD	GLPostingAdapter	AccountAdapter, BatchEodAdapter — fires only for Branch A

Appendix D — Consolidated Feature Flag Registry

Per the constraint, every flag gating money movement carries both an Engineering and a Finance/Risk owner. Flags marked "No" under Money Movement require Engineering ownership only.

Flag Key	Epic	Money Movement?	Engineering Owner	Finance/Risk Owner
loan.booking.enabled	CB-BOOK	No	Loan Subledger lead	Not required — epic moves no money
loan.disbursement.enabled	CB-DSB	Yes	Loan Subledger lead	Treasury Ops lead
loan.accrual.batch.enabled	CB-ACCR	Yes	Batch/EOD lead	Controller / Financial Reporting owner
loan.repayment.autopay.enabled	CB-REPAY	Yes	Loan Subledger lead	Cash Ops/Treasury lead
loan.delinquency.feesassessment.enabled	CB-DELINQ	Yes	Loan Subledger lead	Collections/Risk lead
loan.delinquency.feesposting.enabled	CB-DELINQ	Yes	Loan Subledger lead	Collections/Risk lead
loan.payoff.quote.enabled	CB-PAYOFF	No	Loan Subledger lead	Not required — quote step posts nothing
loan.payoff.settlement.enabled	CB-PAYOFF	Yes	Loan Subledger lead	Treasury/Controller
loan.chargeoff.enabled	CB-CHARGEOFF	Yes	Loan Subledger lead	Controller / Allowance-for-Losses owner
loan.modification.enabled	CB-MOD	No	Loan Subledger lead	Not required for the Branch-B path
loan.modification.configoptimization.enabled	CB-MOD	Yes	Loan Subledger lead	Controller